

Rivage: A Polyglot Distributed Compute Engine for Numerical and Batch Processing

A Report submitted
in partial fulfillment of the requirement
for the award of a Degree of

Bachelor of Technology

in

Computer Science and Engineering

by

Your Name

(Your Roll Number)

Under the guidance of

Guide Name

**Department of Computer Science and Engineering
National Institute of Technology, Srinagar,
Kashmir 190006, INDIA**

May 2026

CHAPTER 0

CERTIFICATE

[To be drafted: A rigorous certification of original academic work, signed and formally verified by the supervising faculty in strict accordance with the NIT Srinagar format mandates.]

CHAPTER 0

STUDENT DECLARATION

[To be drafted: A binding declaration confirming absolute original authorship and guaranteeing the total absence of plagiarized material.]

CHAPTER 0

ACKNOWLEDGEMENTS

[To be drafted: A professional acknowledgement extending appropriate gratitude to the supervisor, the departmental faculty, and the hosting institution.]

CHAPTER 0

ABSTRACT

[To be drafted: A high-density summary of the architectural claims. This must emphasize the zero-dependency Go orchestrator, the polyglot POSIX boundaries, and the deterministic fault-tolerance mechanisms that define the system.]

CHAPTER 0

TABLE OF CONTENTS

CERTIFICATE	III
STUDENT DECLARATION	IV
ACKNOWLEDGEMENTS	V
ABSTRACT	VI
LIST OF FIGURES	IX
LIST OF TABLES	X
ABBREVIATIONS / NOTATIONS / NOMENCLATURE	XI
1 INTRODUCTION	1
1.1 Overview	1
2 Literature Review	3
2.1 Evolution of Distributed Computation Paradigms	3
2.2 Middleware Tax and Polyglot Limitations	3
2.3 Fault Tolerance and State Recovery	4
2.4 Resource Routing and Communication Layers	5
2.5 Theoretical Limits of Distributed Computing	5
3 PROPOSED METHODOLOGY	7
3.1 Problem Statement	7
3.2 Problem Description and System Requirements	7
3.3 Background and Theoretical Foundations	9
3.4 System Architecture & Design	18
3.5 The Coordinator (Control Plane)	22
3.6 The Scheduler (Resource Allocation and Load Balancing)	30
3.7 The Worker Daemon	37
3.8 Fault Tolerance and System Resilience	43
3.9 The Programming Model and Programmable Shuffles	46

4	RESULTS AND DISCUSSION	50
5	CONCLUSION AND FUTURE WORKS	51
	REFERENCES	52
A.	APPENDIX A: Output	53

CHAPTER 0

LIST OF FIGURES

Figure 1	Theoretical data flow and functional boundaries of the MapReduce paradigm	12
Figure 2	Execution trace of a distributed Word Count workload. Network shuffle intersections are rendered cleanly using crossing halos.	15
Figure 3	Pseudocode representation of topological graph resolution via Kahn's Algorithm	17
Figure 4	Vertical topology of the Rivage Engine. Both the orchestration layer and the worker subprocess pipelines flow perfectly top-to-bottom.	19
Figure 5	The internal task state machine managed by the Coordinator.	27
Figure 6	Pseudocode representation of the asynchronous batched state evaluation loop (<code>watchJob</code>)	29
Figure 7	Execution flow and decision matrix of the Least-Loaded routing algorithm.	34
Figure 8	Part 1: The $O(N)$ filtering matrix isolating hardware constraints and maximizing data locality.	35
Figure 9	Part 2: Applying the least-loaded heuristic and physical CPU tie-breakers to select the final node.	36
Figure 10	The Worker Daemon's bounded-memory I/O pipeline. Data is continuously flushed to local disk to preserve RAM during polyglot execution.	40
Figure 11	Pseudocode representing the bounded-memory POSIX execution pipeline within the Worker Daemon.	42
Figure 12	Pseudocode representing the deterministic WAL recovery algorithm executed during master node reboot.	45

CHAPTER 0

LIST OF TABLES

Table 1	Concurrency primitives utilized to prevent network bottlenecking and thread starvation.	24
Table 2	Analytical comparison of implemented routing algorithms and their theoretical complexities.	37
Table 3	Failure matrix detailing deterministic recovery actions across the Compute and Control planes.	46

CHAPTER 0

ABBREVIATIONS / NOTATIONS / NOMENCLATURE

[To be drafted: An exhaustive index of technical abbreviations utilized throughout the text (e.g., DAG, RPC, WAL, gRPC, OOM, mTLS).]

CHAPTER 1

INTRODUCTION

1.1 Overview

The physical constraints of single-node architectures have long precipitated a crisis in systems engineering. As computational demands scale exponentially against the rigid ceilings of localized CPU and GPU capacities, the inevitable bottleneck of von Neumann architectures becomes painfully apparent. A solitary machine, regardless of its aggressive hardware provisioning, is ultimately bound by thermal limits, bus bandwidth, and physical die constraints. We are therefore compelled to look outward. Distributed computation—the orchestration of disparate, geographically scattered hardware into a cohesive logical unit—is not merely an architectural preference. It is a mathematical necessity.

Industrial solutions to this computational deficit have, predictably, ossified into monolithic giants. Systems such as Apache Hadoop dominate the enterprise space, yet they bring with them an architecture that is overwhelmingly complex, notoriously difficult to provision, and inextricably tethered to the Java Virtual Machine (JVM). Such structural rigidity exacts a heavy toll. It imposes severe memory footprints, demands steep operational learning curves, and actively resists the integration of lightweight, polyglot scripting unless mediated by highly inefficient inter-process wrappers. Deploying this scale of infrastructural mass for general-purpose batch processing is akin to using a sledgehammer to crack a walnut. It is fundamentally over-engineered.

In direct response to this operational bloat, this manuscript formally details the architecture of Rivage: a lightweight, high-throughput MapReduce orchestration engine engineered specifically for mechanical agility. We constructed this system entirely from the ground up using Go (Golang). The selection of Go was highly intentional, capitalizing upon its native stackful coroutine concurrency model and exceptionally low-latency

1 | Department of Computer Science and Engineering, National Institute of
Technology Srinagar

networking primitives. The architectural focus centers squarely on bounded-memory I/O, utilizing HTTP/2 multiplexing via gRPC and strict binary contracts established through Protocol Buffers.

The defining characteristic of the engine is its unapologetic polyglot execution environment. By treating worker daemons strictly as agnostic execution vessels, we completely isolate the orchestration mechanics from the actual computational logic. Data serialization relies upon a precise mixture of structured JSON and raw binary formats, interfacing with a centralized disk-backed state store to guarantee durability. Consequently, developers can implement map and reduce transformations natively in the language of their choosing—Python, Rust, C++, or standard shell scripts—while the Go-based control plane maintains absolute, centralized authority over network routing and cluster synchronization.

CHAPTER 2

Literature Review

2.1 Evolution of Distributed Computation Paradigms

Dean and Ghemawat fundamentally redefined the theoretical boundaries of distributed computation with their formalization of the MapReduce paradigm [1]. Their architectural model elegantly abstracted the terror of network topology away from the developer. It pioneered native data parallelization, automatic fault tolerance, and strict data locality—allowing engineers to orchestrate work in a purely value-oriented manner. Yet, the original implementation was not without severe physiological flaws. Specifically, it suffered from a debilitating disk I/O bottleneck. Because the orchestration logic demanded hyper-resilient state checkpoints, the complete output of the map phase had to be written sequentially to physical platters before the shuffle phase could even commence. This incurred an astronomical latency penalty.

Zaharia et al. later addressed this mechanical inefficiency by introducing Apache Spark, built entirely around the abstraction of Resilient Distributed Datasets (RDDs) [2]. This intervention forced a systemic pivot toward in-memory data caching—effectively bypassing the disk I/O penalty by retaining intermediate computational states within volatile RAM. While mathematically elegant, this shift traded disk latency for massive memory saturation, an exchange that is not always strictly beneficial in resource-constrained environments.

2.2 Middleware Tax and Polyglot Limitations

The hegemony of enterprise-grade frameworks—chiefly Hadoop and Spark—has successfully solved planetary-scale distributed processing, primarily through heavy reliance upon the JVM and sprawling dependency graphs. Such density is entirely justified for massive corporate clusters. For the vast majority of mid-tier computational workloads, however, it imposes an unacceptable middleware tax. Operators are forced

to provision gigabytes of idle RAM simply to maintain the runtime environment. Compounding this structural inefficiency is a rigid linguistic constraint. Attempting to execute non-JVM tasks requires fragile, serialized inter-process communication that frequently collapses under load, rendering the deployment of minor, language-agnostic distributed workloads an exercise in profound frustration.

2.3 Fault Tolerance and State Recovery

Hardware volatility is not a remote possibility in distributed computing; it is an absolute statistical certainty. Components will fail. The architecture must anticipate network partitions, memory faults, and sudden power losses. We can broadly categorize these inevitable failures across two distinct axes of statefulness:

- **Stateless Compute (Worker Nodes):** Worker nodes, operating within a decoupled compute plane, are fundamentally expendable. Because these daemons act as completely interchangeable execution engines, they store no permanent cluster state. Recovering from a worker crash is therefore a trivial matter of graph logic. Assuming tasks enforce strict idempotency, the orchestrator simply abandons the interrupted process and re-allocates the workload to a surviving node. This specific fault-tolerance pattern—relying on pure functional repetition rather than complex state rollback—was initially popularized by the original MapReduce framework [1].
- **Stateful Compute (Master Node/Control Plane):** The central orchestrator represents a far more precarious point of failure. A master node cannot merely be rebooted from a blank slate; doing so would annihilate the active Directed Acyclic Graph (DAG) topology and the execution progress of the entire cluster. To survive a catastrophic control plane crash, the system is mandated to record every state mutation to a highly durable log prior to executing the actual network dispatch. This technique, the Write-Ahead Log (WAL), was rigorously formalized by Mohan et al. in the ARIES algorithm [3]. It provides the mathematical guarantee that the master can deterministically reconstruct its precise operational state following a total power loss.

2.4 Resource Routing and Communication Layers

Birrell and Nelson established the theoretical foundation for cross-network procedural communication with the introduction of the Remote Procedure Call (RPC) [4]. Legacy cluster managers have clung to heavily modified, internal RPC implementations for decades, often resulting in severe TCP bottlenecks. Modern protocols like gRPC radically refine this approach by operating exclusively over HTTP/2 and utilizing Protocol Buffers for strict serialization [5]. Instead of transmitting bloated, monolithic payloads, gRPC establishes bounded-memory binary streams. This specific architectural shift makes highly concurrent telemetry and task dispatching exponentially more efficient. The binary contract is rigid, aggressively optimized, and strips away infrastructure overhead. Consequently, a lightweight orchestration engine can effortlessly sustain massive throughput and cluster synchronization without succumbing to the memory bloat typical of older cluster managers.

2.5 Theoretical Limits of Distributed Computing

Any serious evaluation of a distributed processing engine must first acknowledge the immutable mathematical boundaries governing parallel execution. Paradigms such as MapReduce are not exempt from the physical laws of network transport, nor can they evade the formal constraints established by Amdahl's Law.

2.5.1 Amdahl's Law and the Sequential Bottleneck

Amdahl formally articulated the theoretical maximum speedup of any computational task when distributed across parallel processing units [6]. Let p designate the strict proportion of the workload capable of absolute parallelization, and N represent the total volume of active compute nodes. The total system speedup, denoted as $S(N)$, is defined thus:

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}}$$

As the variable N approaches infinity, the theoretical speedup ceiling converges violently toward $\frac{1}{1-p}$. This equation serves as a sobering reminder. The ultimate

performance of a distributed cluster is irreversibly gated by its strictly sequential, unparallelizable operations.

Within MapReduce environments, the isolated Map and Reduce stages naturally exhibit near-perfect parallelism, drastically increasing the value of p [1]. The intermediate Network Shuffle, however—the barrier where all nodes must suspend computation to route and aggregate keys—constitutes a massive, rigid sequential block. The entire engineering effort of a modern distributed system must, therefore, be intensely directed toward optimizing network transport protocols to compress this sequential fraction to its absolute minimum.

CHAPTER 3

PROPOSED METHODOLOGY

3.1 Problem Statement

The objective of this research is to architect and implement a lightweight distributed compute engine capable of orchestrating Directed Acyclic Graph (DAG) workflows alongside native MapReduce tasks across a highly heterogeneous hardware network. We advance a decoupled Master-Worker topology, programmed exclusively in Go, explicitly to eliminate the operational nightmare of legacy cluster management. The resulting system incorporates bounded-memory gRPC streams to guarantee high-throughput communication, native operating system piping to facilitate polyglot task execution, and a strict Write-Ahead Log (WAL) to ensure deterministic control-plane recovery. By systematically excising heavy Java Virtual Machine (JVM) middleware and rigid architectural dependencies, this work seeks to democratize cluster computation. We deliver a mathematically sound, language-agnostic framework that scales linearly upon standard, off-the-shelf commodity hardware.

3.2 Problem Description and System Requirements

To successfully abstract the severe complexities of topological routing and state synchronization away from the end-user, the proposed system must present a rigid, predictable interface. The core problem requires formalizing a declarative environment where developers define execution dependencies without touching a single network socket.

3.2.1 Workflow Formalization (The DAG Requirement)

Execution workflows require strict mathematical modeling as a Directed Acyclic Graph (DAG), denoted formally as $G = (V, E)$:

- **Vertices (V):** Every vertex operates as a strictly bounded, independent computational task. The system is required to allow definition of a vertex through the minimal provision of an executable path—be it a compiled C++ binary or an interpreted Python script—alongside its operational arguments.
- **Edges (E):** Directed edges strictly enforce execution barriers and data provenance. A directed edge originating at task A and terminating at task B constitutes a systemic guarantee: task B is physically barred from execution until task A has unambiguously resolved and its resultant output is persisted to disk.

3.2.2 Universal Data Boundaries

Accommodating multiple programming languages without inflicting framework-specific SDKs upon the user demands the enforcement of a universal, primitive data boundary. The interface bridging the Go orchestrator and the polyglot subprocess must respect the following invariant constraints:

- **Ingress Channel (Input):** The engine must supply a standardized mechanism to stream theoretically unbounded datasets directly into the task’s memory space. The mandated data format must remain highly predictable and sequentially parsable—such as line-delimited JSON or strict binary framing—ensuring the localized script can consume the stream sequentially without triggering kernel memory exhaustion.
- **Egress Channel (Output):** A language-agnostic egress channel is absolutely necessary for tasks to yield their computed results. The system must continuously intercept this output, safely writing it to disk and routing it toward the subsequent pipeline phase, all without requiring the user’s code to initiate a network connection.
- **Fault and Telemetry Channel:** We require an isolated telemetric boundary strictly reserved for error propagation. A task must possess a deterministic method to signal a fatal execution error, immediately alerting the orchestrator to sever the data flow, halt the stage, and aggressively prevent the downstream pollution of corrupted intermediate states.

3.2.3 Execution Constraints (Idempotency)

The volatile nature of distributed hardware guarantees that worker daemons will occasionally drop off the network mid-execution. A resilient orchestrator must detect this and rapidly reassign the aborted task to a surviving node.

This non-negotiable retry mechanism heavily restricts the user's programming model: every submitted task must be strictly idempotent. A developer cannot rely on the persistence of local file systems, nor can they depend upon cross-invocation memory states. If the orchestrator feeds an identical dataset into a task, the script must mathematically yield the exact same output, regardless of whether the cluster has been forced to restart that specific task ten times due to rolling hardware faults.

3.2.4 Deployment Requirements

Finally, to resolve the immense friction inherent in traditional enterprise data tools, the physical deployment of the engine must remain trivial.

- The architecture absolutely must not rely upon external cluster managers, nor demand the presence of the Java Virtual Machine.
- Compilation must yield a single, standalone static binary. An operator must be able to instantly designate a given machine as either the control plane or a compute plane using nothing more than primitive command-line flags.
- Worker nodes require the ability to autonomously discover and authenticate with the master node at runtime, entirely bypassing the need for statically configured, fragile network routing tables.

3.3 Background and Theoretical Foundations

Before dissecting the physical topology of the proposed engine, we must solidify the theoretical framework that dictates data mutation across a distributed network. The MapReduce computing paradigm derives its structural integrity from functional programming, specifically adapting the `map` and `fold` (or `reduce`) primitives prevalent in strict languages like Lisp and Haskell. The inherent mathematical purity of these functions allows the orchestrator to blindly parallelize workloads across thousands of discrete machines with absolute safety.

3.3.1 Pure Functions and Side Effects

A rigorous understanding of pure functions is required here. A function achieves this classification only if it adheres to two inflexible rules:

- **Deterministic Output:** Given a specific input x , the evaluation of $f(x)$ will invariably yield the exact identical output y . The physical location of the hardware, the time of day, and the state of the network have zero bearing on the result. Impure functions inherently violate this by relying on external entropy or global states.
- **No side-effects:** The function is strictly prohibited from mutating any external state. It cannot write to a shared database, increment a global network counter, or alter a variable outside its immediate lexical scope.

In an environment characterized by systemic hardware failure, this mathematical purity is our primary defense mechanism. Should a worker node crash midway through a task, the master must re-allocate the work. If the user's code contained side effects—perhaps partially committing a transaction before the power loss—restarting the task would permanently corrupt the global cluster state. By forcefully constraining the Map and Reduce phases to pure functional boundaries, the engine natively achieves true idempotency. A panicked task is safely abandoned. The system guarantees exactly-once processing semantics without ever relying upon Byzantine distributed rollback protocols.

3.3.2 Mathematical Formulation of the Paradigm

We process data fundamentally as an infinite stream of key-value tuples. The execution pipeline can be formalized by observing the strict domain and codomain mappings of its three isolated phases: **Map, Shuffle, and Reduce**.

1. **The Map Function:** The map operation ingests a singular key-value tuple and emits a list of intermediate tuples. Formally, if K denotes the set of keys and V the set of values, the signature demands:

$$\text{Map} : (K_1 \times V_1) \rightarrow \text{List}(K_2 \times V_2)$$

Because the function evaluates entirely in isolation upon a single (K_1, V_1) pair, the Coordinator has the mathematical authority to slice the dataset into arbitrary physical

partitions, distributing them across the cluster. This phase guarantees trivial $O(N)$ horizontal scalability.

2. **The Shuffle and Group-by-key Barrier:** The conclusion of the map phase forces a cluster-wide synchronization barrier. The scattered intermediate pairs require aggressive network routing to guarantee that every value sharing a common K_2 key converges at a single physical destination. This system-level aggregation is modeled thus:

$$\text{Shuffle} : \text{List}(K_2 \times V_2) \rightarrow \text{List}(K_2 \times \text{List}(V_2))$$

This barrier represents the most computationally hostile operation in a distributed network, demanding an intense, all-to-all network exchange to successfully align the data for the final phase.

3. **The Reduce Function:** Following the shuffle resolution, the system invokes the user's defined fold operation. It receives a unique K_2 key paired with its comprehensively grouped list of values, executing a deterministic aggregation:

$$\text{Reduce} : (K_2 \times \text{List}(V_2)) \rightarrow \text{List}(V_3)$$

By confining all stateful aggregation strictly to the post-shuffle Reduce phase, the orchestrator perfectly shields the user from the horrors of distributed concurrency.

3.3.3 Theoretical Execution Flow

The visual representation in Figure 1 illustrates this rigid functional pipeline. By adhering to this mathematical model, the master node remains blissfully ignorant of the user's internal script logic. It operates solely to fulfill the geometric contract of the data signatures.

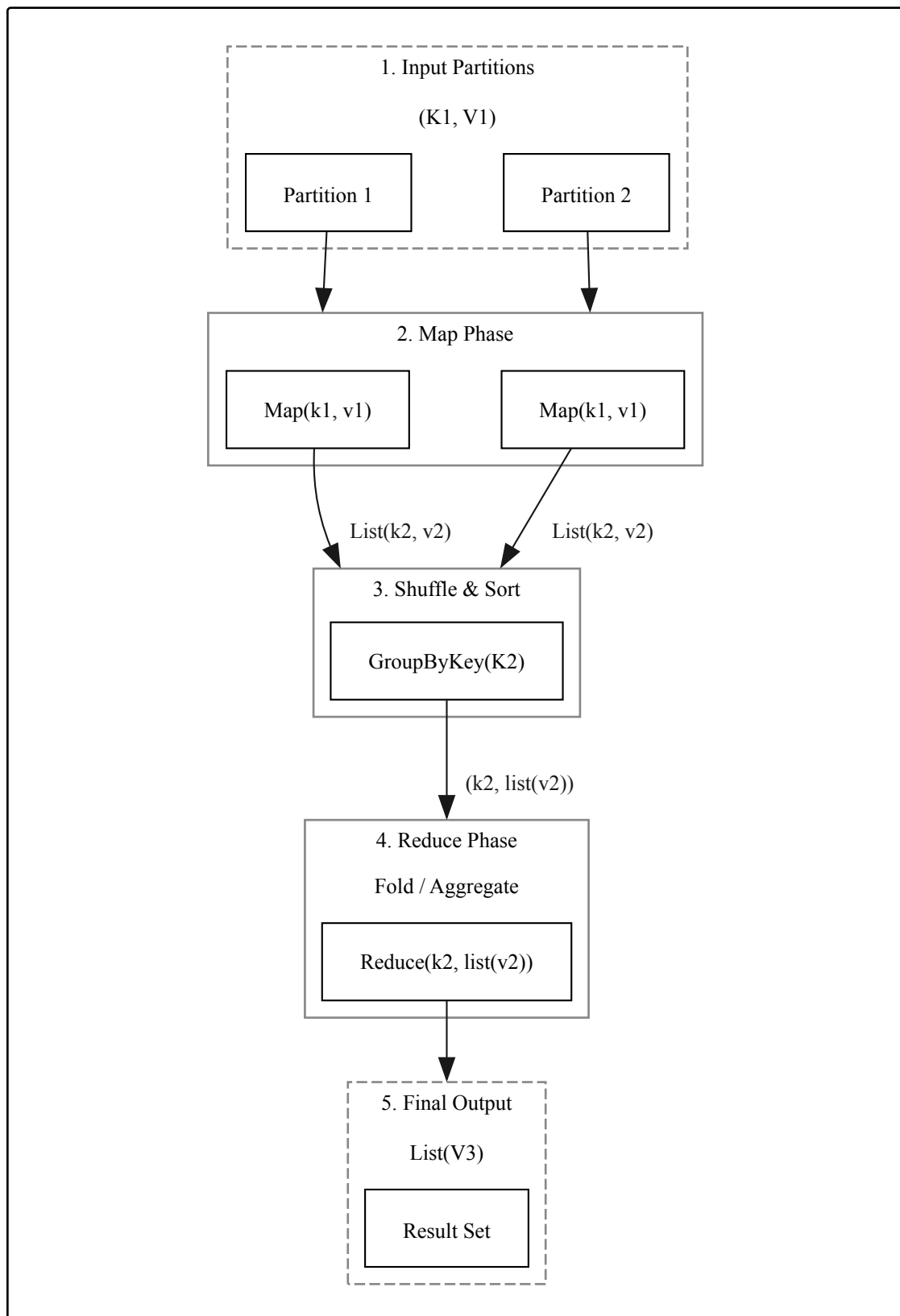


Figure 1: Theoretical data flow and functional boundaries of the MapReduce paradigm

3.3.4 Applied Example: Distributed Word Frequency Analysis

Contextualizing these abstract mathematical formulations requires tracing a physical dataset through the functional pipeline. We examine the canonical distributed workload: word frequency analysis, wherein the cluster calculates the precise occurrence of every distinct token across a massive, unstructured text corpus.

Assume the Central Store maintains a raw dataset. The Coordinator fragments this payload, pushing it out to two distinct Worker daemons. The data crosses the functional boundaries as follows:

1. **Input Partitioning:** The system cleaves the raw byte stream into two bounded partitions, transmitting them via gRPC:
 - Partition A: "hello world hello"
 - Partition B: "hello distributed compute"
2. **The Map Phase:** The isolated script ingests the stream. For each token parsed, it yields an intermediate tuple containing the word and the integer 1. Operating as a pure function, the nodes possess no awareness of one another, requiring no distributed mutex locks to perform their local work.
 - Node 1 (Processing Partition A) emits:
("hello", 1), ("world", 1), ("hello", 1)
 - Node 2 (Processing Partition B) emits:
("hello", 1), ("distributed", 1), ("compute", 1)
3. **The Shuffle and Group Barrier:** The engine aggressively intercepts these raw tuples, initiating the all-to-all network synchronization barrier. It collates identical keys, guaranteeing that every instance of a specific token is physically routed to the same destination reducer.

Grouped Output:

- "hello" → [1, 1, 1]
- "world" → [1]
- "distributed" → [1]
- "compute" → [1]

4. The Reduce Phase:

The user's reduce script receives the mathematically grouped lists. It executes a rudimentary sum fold over the integers, generating the final, deterministic totals.

- `Reduce("hello", [1, 1, 1]) → ("hello", 3)`
- `Reduce("world", [1]) → ("world", 1)`
- `Reduce("distributed", [1]) → ("distributed", 1)`
- `Reduce("compute", [1]) → ("compute", 1)`

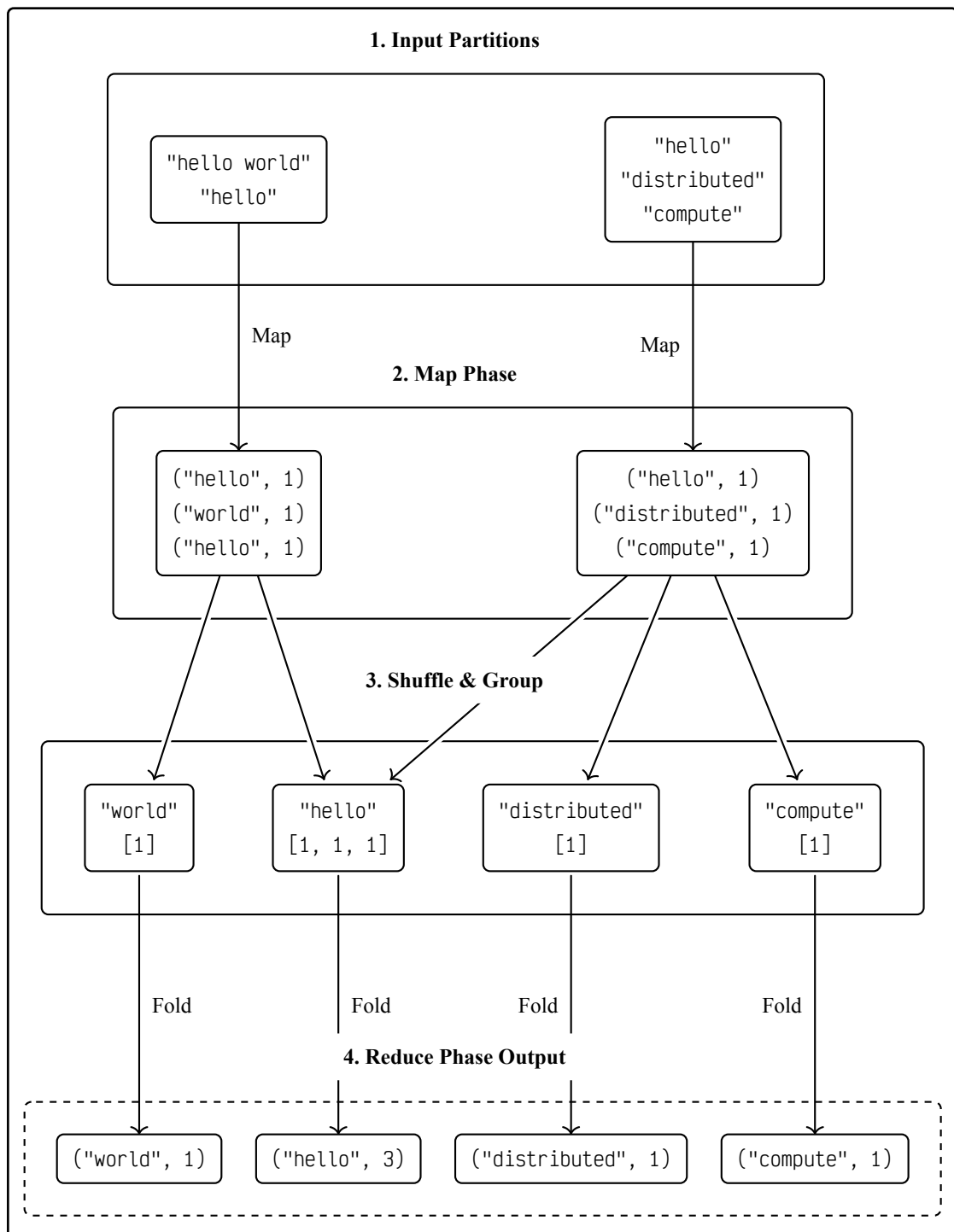


Figure 2: Execution trace of a distributed Word Count workload. Network shuffle intersections are rendered cleanly using crossing halos.

By isolating the token-counting logic behind these stateless functional barriers, the engine scales natively. The end-user writing the Python or C++ logic is entirely spared from managing race conditions, debugging mutex deadlocks, or attempting to safely increment a shared variable over a volatile network.

3.3.5 Graph Theory and Directed Acyclic Graphs (DAG)

To orchestrate complex computational workflows, we must formally model the execution topology using Graph Theory, specifically defining it as a Directed Acyclic Graph (DAG), expressed as $G = (V, E)$:

- **Vertices (V):** Every vertex operates as a strictly bounded, independent computational task.
- **Edges (E):** The directed edges dictate absolute data flow dependencies. An edge $(u, v) \in E$ is a mathematical guarantee: task v is paralyzed and cannot be scheduled until task u has reached terminal success and relinquished its outputs.

The “acyclic” constraint is not merely a suggestion; it is structural law. The graph must contain zero directed cycles. It must be physically impossible to traverse a sequence of edges starting at v and returning to v . In distributed orchestration, a cycle mathematically guarantees a catastrophic infinite deadlock, permanently stalling the cluster.

3.3.6 Topological Sorting

To map a multi-dimensional graph onto sequential hardware, the DAG requires algorithmic flattening. This process, Topological Sorting, resolves the two-dimensional topology into a linear execution queue that strictly honors all declared dependencies.

Input: A Directed Acyclic Graph $G = (V, E)$

Output: A linear list L containing the topologically sorted vertices

```

1:  $L \leftarrow$  Empty list that will contain the sorted elements
2: InDegree  $\leftarrow$  Array storing the number of incoming edges for each  $v \in V$ 
3:  $Q \leftarrow$  Queue containing all nodes where  $\text{InDegree}[v] == 0$ 
4:
5: while  $Q$  is not empty do
6:    $u \leftarrow$  Dequeue a node from  $Q$ 
7:   Append  $u$  to  $L$ 
8:
9:   for each node  $v$  with a directed edge from  $u$  to  $v$  do
10:      $\text{InDegree}[v] \leftarrow \text{InDegree}[v] - 1$  // Relax the edge
11:     if  $\text{InDegree}[v] == 0$  then
12:       Enqueue  $v$  into  $Q$ 
13:
14: if length of  $L \neq |V|$  then
15:   return Error (Graph contains at least one cycle)
16: else
17:   return  $L$ 

```

Figure 3: Pseudocode representation of topological graph resolution via Kahn's Algorithm

A valid topological sort ensures that for every directed edge (u, v) , node u invariably precedes node v in the final execution array. This operation is non-negotiable. It structurally isolates independent vertices into distinct, parallelizable stages separated by rigid synchronization barriers. Intermediate data is thereby forced into a completely settled, immutable state before any dependent downstream process is permitted to read it.

3.3.7 Network Transport Theory: RPC and HTTP/2 Multiplexing

Birrell and Nelson’s original Remote Procedure Call (RPC) was a revelation, designed to mask network communications as localized function calls [4]. Unfortunately, legacy RPC and standard REST protocols operating over HTTP/1.1 suffer immensely from TCP Head-of-Line (HoL) blocking. A single dropped packet in a sequential queue paralyzes the entire TCP connection, inducing severe latency spikes.

Modern infrastructure entirely bypasses this flaw by demanding HTTP/2 transport [5]. The introduction of binary framing and aggressive stream multiplexing permits hundreds of independent RPC calls to interleave concurrently over a solitary, persistent TCP socket. This architecture eradicates HoL blocking, minimizes TLS handshake overhead, and yields the highly responsive, full-duplex communication channels absolutely required for real-time cluster telemetry.

3.4 System Architecture & Design

To systematically satisfy the rigid operational constraints established in the problem statement, the engine deploys a strict Master-Worker topology written exclusively in Go. This architecture forcefully isolates the Control Plane—which handles global state mapping, dynamic scheduling, and network multiplexing—from the Compute Plane, which exists solely to execute isolated POSIX processes.

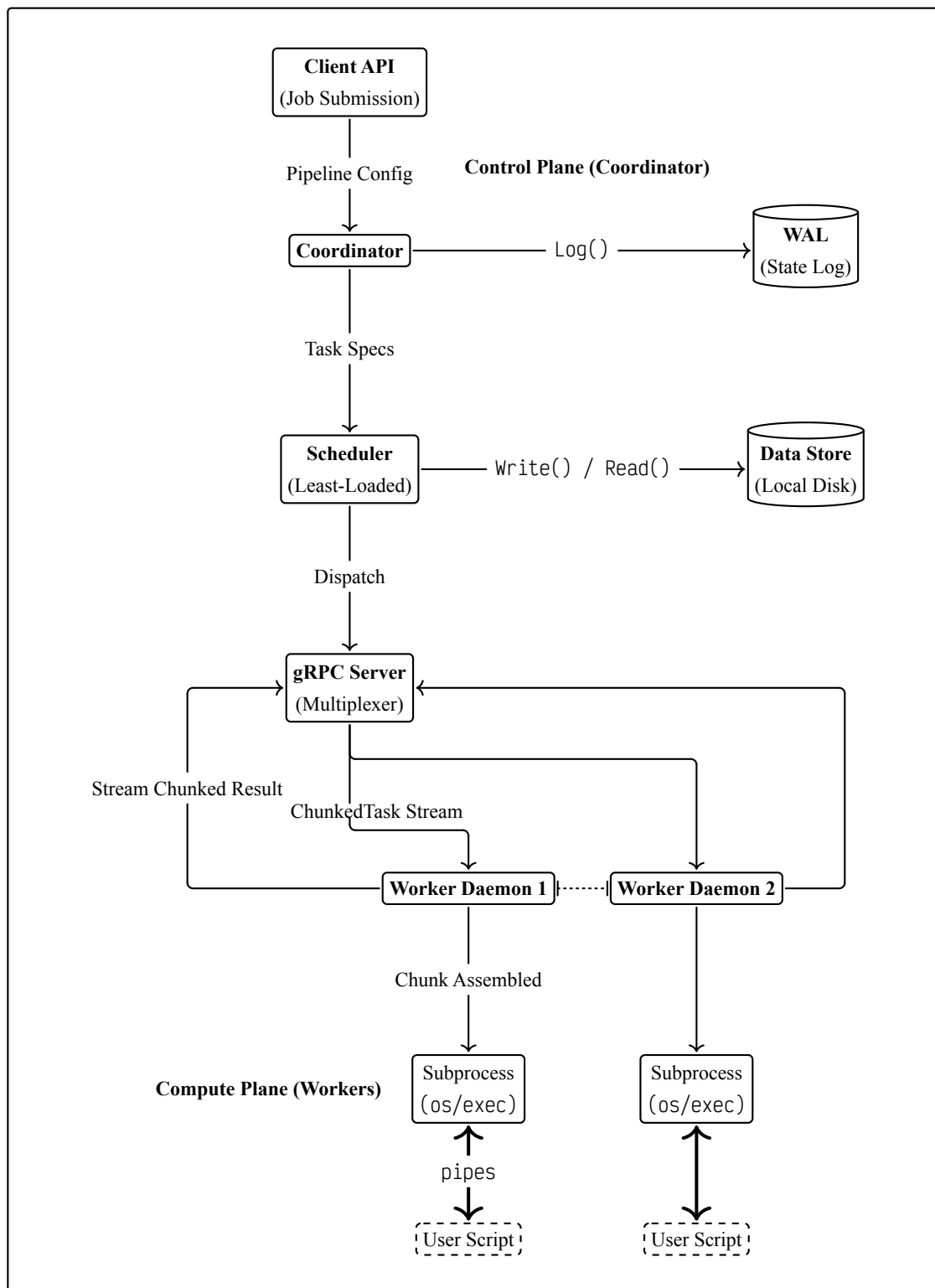


Figure 4: Vertical topology of the Rivage Engine. Both the orchestration layer and the worker subprocess pipelines flow perfectly top-to-bottom.

The topology outlined in Figure 4 operates not just as a physical layout, but as an immutable sequence of data transformations. Tracing the lifecycle of a massive dataset through these network boundaries reveals exactly how the orchestrator manages ingestion, state synchronization, and bounded I/O streaming.

3.4.1 The Control Plane and DAG Resolution

The lifecycle initiates at the Coordinator. When the Client API ingests a MapReduce job, the payload formally defines the workflow utilizing the `dag.Builder` interface. The Coordinator instantly routes this payload into the static topological analyzer, executing Kahn’s algorithm to mathematically flatten the graph into a strict, acyclic `pipeline.Order` array.

During the initial source stage, the Coordinator aggressively cleaves the raw, monolithic dataset into manageable `TaskInput` structs. These fragments are encased within a `Protocol Buffer TaskSpec` contract—a rigid binary definition dictating the exact executable path, the necessary POSIX environment variables, and the specific hardware tags required for scheduling.

3.4.2 State Persistence and the Write-Ahead Log (WAL)

Because the Coordinator maintains the global execution state entirely within volatile memory, it represents a dangerous single point of failure. A master node kernel panic destroys the cluster.

To guarantee fault tolerance while avoiding the crippling latency of external database transactions, we implemented a localized Write-Ahead Logging (WAL) mechanism. Before a single byte of a `TaskSpec` traverses the network, the Coordinator forces a deterministic state transition event (e.g., `walEventDispatched`) to a binary `.wal` file on the NVMe disk. Concurrently, it provisions an isolated directory within the `ResultStore` to capture incoming computations. Should the master node suffer a catastrophic power failure, the `RecoverEntries` routine reads this binary log sequentially, perfectly reconstructing the exact graph state, ignoring completed tasks, and instantly re-queuing pending workloads.

3.4.3 Scheduler Allocation and Optimistic Locking

The asynchronous `watchJob` driver scoops up these pending tasks. The Scheduler evaluates the Compute Plane by analyzing a detached snapshot of real-time worker telemetry. To violently prevent network race conditions—a scenario where the master node fires thousands of payloads at a single worker before the daemon’s CPU gauge has a chance to update—the Scheduler relies on optimistic locking. The Coordinator tracks a local `inFlight` atomic integer alongside the reported load. The Scheduler’s core heuristic—whether utilizing Least-Loaded or Tag-Affinity matrices—incorporates this atomic variable to structurally guarantee that physical node capacities are never over-subscribed.

3.4.4 Network Transport and Bounded-Memory Chunking

Upon securing a worker allocation, the Coordinator begins the physical network transfer. Shoving monolithic arrays—like a 2GB log file or massive matrix tiles—into standard RPC buffers reliably triggers immediate Out-Of-Memory (OOM) kernel panics.

To mathematically bound memory complexity to $O(1)$, the system forces a custom chunking pipeline across the multiplexed HTTP/2 streams. The Coordinator slices the payload into rigid 4MB segments, pushing them asynchronously as `ChunkedTask` streams. The receiving Worker Daemon intercepts this stream using an `inboundAssembler`. Rather than accumulating the 4MB slices in RAM, it flushes the bytes directly to a temporary file on the local disk. This bounded I/O mechanism allows cheap, commodity hardware to process datasets exponentially larger than their physical memory capacity.

3.4.5 Polyglot Execution via POSIX Boundaries

Once the binary file is fully settled on the worker’s disk, the data crosses into user space. The Worker Daemon is intellectually blind; it does not parse or comprehend the payload. It simply leverages Go’s `os/exec` library to fork an isolated operating system subprocess containing the user’s defined script. The daemon links the assembled disk file to the script via the `RIVAGE_INPUT_FILE` environment variable, or pipes it natively into the child process’s standard input (`stdin`). The polyglot script executes its mathematical transfor-

mation and emits the deterministic result directly to standard output (`stdout`), which the daemon simultaneously traps and streams into a local output file.

3.4.6 Upstream Telemetry and Stage Synchronization

Upon process termination, the daemon must return the result. Utilizing a `transfer.ChunkReader`, it slices the generated output file into tight 512KB frames, streaming them back to the master via `ChunkedResult` messages.

The Coordinator catches these frames, piping them immediately into the durable `ResultStore`. It then logs a `walEventCompleted` entry. The worker daemon aggressively purges its temporary directories, instantly reverting to a stateless, idle posture. The Coordinator strictly enforces the stage barrier, paralyzing graph progression until all parallel tasks mathematically report success. Once cleared, the Scheduler extracts these outputs, pushes them through the user's custom `ShuffleFunc`, and generates the payload geometry for the subsequent DAG stage.

3.5 The Coordinator (Control Plane)

The Coordinator is the absolute sovereign of the Rivage architecture. Operating entirely outside the user's computational space, it functions as a highly optimized finite state machine, a network multiplexer, and a strict resource allocator. Because this node alone manages the volatile state of thousands of concurrent tasks, its internal architecture is aggressively engineered to bypass memory saturation and thread starvation.

3.5.1 Core Responsibilities

Its operational directives are fiercely limited to the following:

- **DAG Orchestration and Topological Sorting:** Ingests logical topologies from the Client API. Executes Kahn's Algorithm to mathematically flatten dependencies, structurally blocking downstream task execution until upstream prerequisites resolve flawlessly.
- **Task Generation and Payload Fragmentation:** Slices raw datasets into bounded partitions. Binds these fragments to executable definitions, yielding fully isolated, mathematically independent execution payloads.

- **State Persistence and Disaster Recovery:** Circumvents slow external databases by committing state mutations exclusively to an append-only Write-Ahead Log (WAL). This ensures zero-latency disk writes while mathematically guaranteeing cluster state recovery after a master crash.
- **Scheduler Delegation:** Continuously polls the internal state maps, delegating hardware routing entirely to the Scheduler interface via an optimistic locking algorithm that prevents horrific network congestion.
- **gRPC Multiplexing:** Sustains persistent, full-duplex HTTP/2 TCP streams with every registered worker, enabling simultaneous payload fragmentation and microsecond-accurate hardware telemetry ingestion.

3.5.2 Initialization and Network Configuration

Upon boot, the Coordinator fundamentally rewires the default gRPC server constraints. Standard RPC memory configurations are pitifully inadequate for batch processing. The engine overrides the transport layers:

- **Payload Boundaries:** TCP limits (`grpc.MaxRecvMsgSize` and `grpc.MaxSendMsgSize`) are statically inflated to 16 Megabytes to absorb large matrix tiles without panicking the transport layer.
- **TCP Windowing:** Read and write buffers are expanded to 8 Megabytes, actively preventing network back-pressure during highly parallel chunk streaming.

Concurrently, it provisions an asynchronous HTTP dashboard, utilizing a thread-safe circular ring buffer (`LogRing`) to serve real-time state metrics without ever blocking the critical gRPC event loop. For absolute security, it mandates Mutual TLS (mTLS) authentication. When a daemon attempts to connect, the Coordinator mathematically verifies its HMAC-SHA256 signature before allowing the TCP socket into the trusted pool.

3.5.3 Internal Struct Layout and Concurrency Mechanics

Managing massive network concurrency demands precise memory isolation. A single global mutex lock would instantly throttle the HTTP/2 loop. The Coordinator intelli-

gently fragments its state across lock-free concurrent maps and highly localized mutexes.

3.5.3.1 The Global Root

The root struct maps identifiers directly to heap pointers in $O(1)$ time complexity, allowing background goroutines to fetch state without blocking the network reader.

```
struct Coordinator {
    // Interfaces
    cfg      : CoordinatorConfig
    sched    : SchedulerInterface
    grpc_srv : ProtocolServer
    log      : TelemetryLogger

    // Global Registries
    workers  : ConcurrentMap [WorkerID → worker_state]
    jobs     : ConcurrentMap [JobID → job_state]
    results  : ConcurrentMap [JobID → OutputCache]
}
```

Shared Resource	Primitive / Lock	Granularity	Architectural Justification
Global Worker Pool	<code>sync.Map</code>	Lock-Free	Permits $O(1)$ lookups and Scheduler routing without blocking the multiplexed HTTP/2 network reader.
Worker CPU Usage	<code>atomic.Value</code>	Hardware-Level	Allows perfectly concurrent heartbeat ingestion and payload dispatch without cross-thread lock contention.
Task State Machine	<code>sync.Mutex</code>	Per-Stage	Microscopic localized locking specifically on <code>job.mu</code> prevents cross-stage thread starvation during massive state mutations.
Worker Core Allocation	Semaphore	Per-Worker	Mathematically bounds spawned OS subprocesses to physical CPU cores, entirely eliminating cache thrashing.

Table 1: Concurrency primitives utilized to prevent network bottlenecking and thread starvation.

3.5.3.2 Worker State (Telemetry & Networking)

The `worker_state` struct aggressively utilizes hardware-level atomic operations to ingest high-frequency telemetry without locking the thread.

```
struct worker_state {  
    // Locks  
    mu          : ReadWriteLock // Protects heartbeat timestamp  
    send_mu     : StandardLock  // Serializes outbound gRPC  
    frames  
  
    // Identity  
    id          : String  
    stream      : GrpcConnection  
    last_heartbeat : Timestamp  
  
    // Hardware Telemetry  
    active_tasks : AtomicInteger  
    in_flight    : AtomicInteger  
    cpu_usage    : AtomicFloat  
}
```

Architectural Justification: — a distinction that is easily overlooked — separating the `mu` read-write lock from the `send_mu` lock achieves genuine I/O multiplexing. The system mutates metrics like `cpu_usage` via atomic swap instructions. Consequently, the Coordinator can update the CPU gauge and blast a 4MB binary chunk to the same daemon at the exact same physical millisecond, entirely devoid of cross-thread lock contention.

3.5.3.3 Job and Task State

State mutations lock only the microscopic `job.mu`, ensuring operations remain brutally granular.

```
struct job {
    mu            : StandardLock
    tasks         : Map [TaskID → task_state]
    done_tasks    : AtomicInteger

    // Disk Boundaries
    store         : ResultStorePointer
    wal           : WriteAheadLogPointer
    chunk_assemblers : Map [TaskID → ByteStream]
}

struct task_state {
    spec          : TaskSpecification
    status        : Enum (Pending | Running | Completed | Failed)
    worker_id     : String
    retries       : Integer
}
```

3.5.4 Dynamic Code Shipping and Protocol Buffers

To eradicate the nightmare of configuring shared network file systems (NFS), the engine implements a dynamic `codeCache`. It scans the incoming DAG, reads the user’s raw source code into a byte array, and embeds it directly inside the `pb.TaskSpec` protobuf payload. The system physically ships the logic across the network alongside the dataset, guaranteeing zero pre-configuration on the worker hardware.

The Coordinator dispatches two primary payload types over the gRPC stream:

- `TaskSpec`: Deployed for tiny datasets. Encapsulates binary data, POSIX environment variables, and the script payload in a single frame.
- `ChunkedTaskSpec`: Invoked when data breaches the safe 4MB boundary. The stream is sequentially fragmented, completely eliminating Head-of-Line (HoL) blocking and preserving the master node’s RAM.

3.5.5 Task Lifecycle and WAL Recovery

3.5.5.1 Logical Definition

Logically, a “Task” binds a physical data partition to an isolated OS process. Because we strictly mandate that user scripts act as pure, idempotent functions, the Coordinator exercises the absolute authority to forcefully terminate, duplicate, and re-allocate tasks unconditionally the moment hardware begins to falter.

3.5.5.2 Execution Lifecycle (Finite State Machine)

The execution of a task is governed by a strict finite state machine tracked within the `job.tasks` mapping:

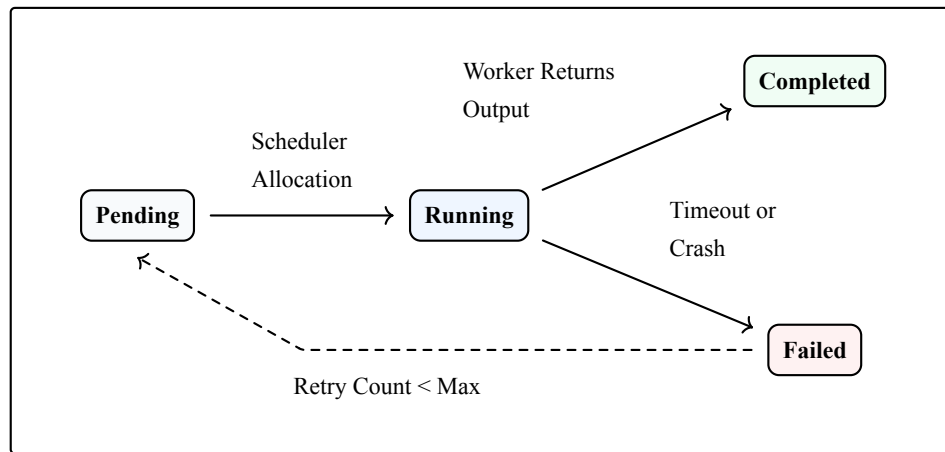


Figure 5: The internal task state machine managed by the Coordinator.

1. **Pending (and Recovery):** The Coordinator scans the WAL. Tasks flagged `walEventCompleted` are instantly pushed to **Completed**. Unfinished garbage is initialized as **Pending**.
2. **Running:** The Scheduler acquires a node. The `in_flight` atomic integer is optimistically incremented, a `walEventDispatched` record hits the disk, and the gRPC binary stream opens.
3. **Completed:** The daemon streams the output back, triggering a localized write to the `ResultStore` and a terminal state mutation.
4. **Failed:** The subprocess panics, or the global watchdog intercepts a timeout. Providing the retry threshold remains intact, the task aggressively reverts to **Pending** for an immediate network re-allocation.

3.5.6 Critical Function Signatures

These internal boundaries are safely abstracted behind a core set of highly optimized routines:

```
// Primary entry point for job ingestion and topology parsing
run_job_raw(ctx Context, job_id String, pipeline Pipeline,
            inputs []TaskInput) → ([]TaskOutput, Error)

// Stage Executor (Invoked iteratively for pipeline.Order)
execute_stage(ctx Context, job_id String, stage Stage,
              tasks []TaskSpec) → ([]TaskOutput, Error)

// Network Dispatcher (Handles Chunking Logic)
dispatch(worker_id String, spec TaskSpec) → Error
```

3.5.7 Orchestration Algorithms

To preserve state integrity without bottlenecking the TCP sockets, the master executes two asynchronous loops: the task driver and the hardware watchdog.

3.5.7.1 The Task Driver (watchJob)

Rather than reacting dynamically to network events (a known trigger for broadcast storms), the system employs an interval-based driver.

Input: Active Job object J , Scheduler S , Write-Ahead Log WAL, Clock Ticker T

Output: Continuous network dispatch of pending tasks in J

```

1: loop
2:   wait for tick from  $T$ 
3:    $P \leftarrow J.\text{snapshotPending}()$     // Safely extract all Pending tasks
4:   if  $P = \emptyset$  then
5:     continue
6:
7:   for each task  $t \in P$  do
8:      $W \leftarrow S.\text{Pick}(t.\text{RequiredTags})$ 
9:     if  $W = \text{null}$  then
10:      break    // No workers available, retry next tick
11:
12:    lock  $J.\mu$ 
13:     $t.\text{status} \leftarrow \text{Running}$ 
14:     $t.\text{worker\_id} \leftarrow W.\text{id}$ 
15:     $W.\text{in\_flight}.\text{Add}(1)$     // Optimistically lock worker capacity
16:     $\text{WAL}.\text{Log}(\text{walEventDispatched}, t.\text{id})$ 
17:    unlock  $J.\mu$ 
18:
19:    spawn  $\text{DispatchStream}(W, t)$ 

```

Figure 6: Pseudocode representation of the asynchronous batched state evaluation loop
(watchJob)

3.5.7.2 Hardware Watchdog (reapDeadWorkerTasks)

Because commodity hardware is notoriously volatile, the system must detect silent network partitions. The global watchdog continuously evaluates `last_heartbeat` timestamps.

If the delta between the system clock and the last heartbeat exceeds the tolerance threshold, the Coordinator mathematically declares the node dead. The reaper algorithm directly intervenes, downgrading active tasks to `Pending` and stripping the `worker_id`. The subsequent driver tick seamlessly re-routes the work to a healthy node, preserving exactly-once semantics effortlessly.

3.5.8 Extended Responsibilities: Telemetry and Assembly

Beyond active routing, it manages critical passive ingestion:

1. **Lock-Free Heartbeat Ingestion:** By updating `atomic.Value` fields directly from the incoming `WorkerMessage_Heartbeat` gRPC frames, it provides the Scheduler with microsecond-accurate CPU telemetry without ever touching a mutex lock.
2. **Disk-Backed Byte Stream Assembly:** When the EOF (`IsFinal`) flag arrives on a chunked result stream, the buffered bytes are violently flushed to the disk-backed `ResultStore`. The master node's volatile RAM remains perfectly shielded, allowing it to ingest terabytes of intermediate data without panicking.

3.6 The Scheduler (Resource Allocation and Load Balancing)

Blindly vomiting tasks onto worker nodes mathematically guarantees hardware saturation, network clogging, and catastrophic straggler effects. The overall execution time of a DAG stage is inextricably bound to its single slowest task. Load balancing is not a luxury; it is the fundamental requirement for cluster throughput.

To solve this, Rivage violently decouples the network dispatching logic from the hardware allocation logic. The Coordinator pauses the dispatch pipeline and completely relinquishes routing authority to the Scheduler interface.

3.6.1 Core Responsibilities and Theoretical Challenges

Operating as a synchronous routing engine, the Scheduler attempts to solve a variation of the NP-hard Bin Packing Problem using extremely fast, greedy heuristics. Its primary operational responsibilities are:

- **Constraint Satisfaction (Hardware Filtering):** Filtering out incompatible hardware via strict set theory (e.g., rejecting a node lacking AVX-512 instruction sets when executing a specialized binary).
- **Dynamic Load Balancing:** Routing based on lock-free CPU telemetry and active process counts to discover the least-burdened nodes in the cluster.
- **Data Locality and Affinity:** Preferring nodes that already possess the intermediate data on their local NVMe drives, thereby destroying redundant network transfers.
- **Latency Masking:** Employing prefetch multipliers to intentionally over-subscribe the TCP pipes, successfully hiding the RTT latency of the gRPC handshake.

3.6.2 Interface Design and Memory Isolation

Standard schedulers in monolithic environments are often crippled by read-write lock contention against the central database. The scheduler cannot route a task without locking the state, thereby blocking incoming telemetry.

In Rivage, we isolate the Scheduler entirely. To ensure routing algorithms execute in sub-millisecond time, the Coordinator passes a detached, read-only slice of the cluster state.

```
interface Scheduler {
    // Evaluates telemetry and returns the optimal Worker ID
    Pick(workers: []WorkerSnapshot, req_tags: []String,
        affinity: []String) → String
}

// Read-only, lock-free copy of daemon telemetry.
struct WorkerSnapshot {
    id          : String
    tags        : []String
    active_tasks : Integer
    cpu_usage    : Float
    last_heartbeat : Timestamp
    cached_keys : []String // Local data availability
    capacity     : Integer  // Hardware core limits
}
```

By operating exclusively on this detached `WorkerSnapshot` array, the scoring matrix resolves entirely within the CPU's L1/L2 cache, never blocking incoming telemetry streams.

3.6.3 Algorithmic Implementations

The engine permits hot-swapping scheduling logic depending on the nature of the workload.

3.6.3.1 Round-Robin Allocation (The Baseline)

The Round-Robin algorithm serves as the baseline heuristic. It increments a pointer sequentially. While computationally trivial ($O(1)$), it is willfully blind. It naively assumes all tasks possess identical complexity and all nodes boast equal throughput. In heterogeneous polyglot environments, this inevitably triggers severe straggler penalties.

3.6.3.2 Hardware Filtering and Set Theory Constraint Satisfaction

The Scheduler treats hardware constraints as a strict subset problem. Let R represent the set of required tags defined by the task, and let T_w represent the set of tags reported by a specific worker w . The Scheduler evaluates the boolean intersection:

$$R \subseteq T_w$$

Failure to satisfy this exact subset results in immediate mathematical elimination from the candidate pool before load scoring even begins.

3.6.3.3 The Thundering Herd Problem and Optimistic Locking

The most violent engineering flaw in distributed scheduling is the Thundering Herd phenomenon. A scheduler sees Worker A is empty, assigns 100 tasks sequentially in a millisecond, and massively overwhelms the node before its next telemetry heartbeat can register the load.

We eradicate this via Optimistic Locking. The precise millisecond a task is routed, the atomic `in_flight` counter increments. When generating the next `WorkerSnapshot`, the Coordinator merges this counter directly into the `ActiveTasks` value. The Scheduler perceives this incoming network load instantly, perfectly distributing the payloads despite the physical telemetry lag.

3.6.3.4 Network Latency Masking and Capacity Limits

To keep hardware saturated despite network Round-Trip Time, the Scheduler inflates capacity utilizing a configurable prefetch multiplier (M_p).

The mathematical capacity limit $L_{\max}$ for a given worker w is defined as:

$$L_{\max} = \max(1, \lfloor w.\text{Capacity} \times M_p \rfloor)$$

A multiplier of 1.0 strictly enforces core-limits. A multiplier of 2.0+ intentionally over-subscribes the node, stuffing the local execution queue so the CPU never idles while waiting on the network to deliver the next binary chunk.

3.6.3.5 Dynamic Data Locality and Affinity Routing

Matrix multiplication generates massive intermediate data. If Node A computed a tile during the Map phase, it already has the data resident on its local drive. Routing the subsequent Reduce operation to Node B triggers a horrific network transfer.

We calculate an affinity intersection score for all eligible workers:

$$S_w = |w.\text{CachedKeys} \cap \mathbf{A}|$$

We isolate the absolute maximum score across the cluster, brutally discarding any node that fails to tie for this locality peak before ever applying load heuristics.

3.6.4 Least-Loaded Allocation (The Core Algorithm)

The primary routing engine applies an $O(N)$ sweep to filter tags, isolates the data locality maximums, and selects the node minimizing `ActiveTasks`. Ties are violently broken utilizing the lock-free `CPUUsage` telemetry gauge.

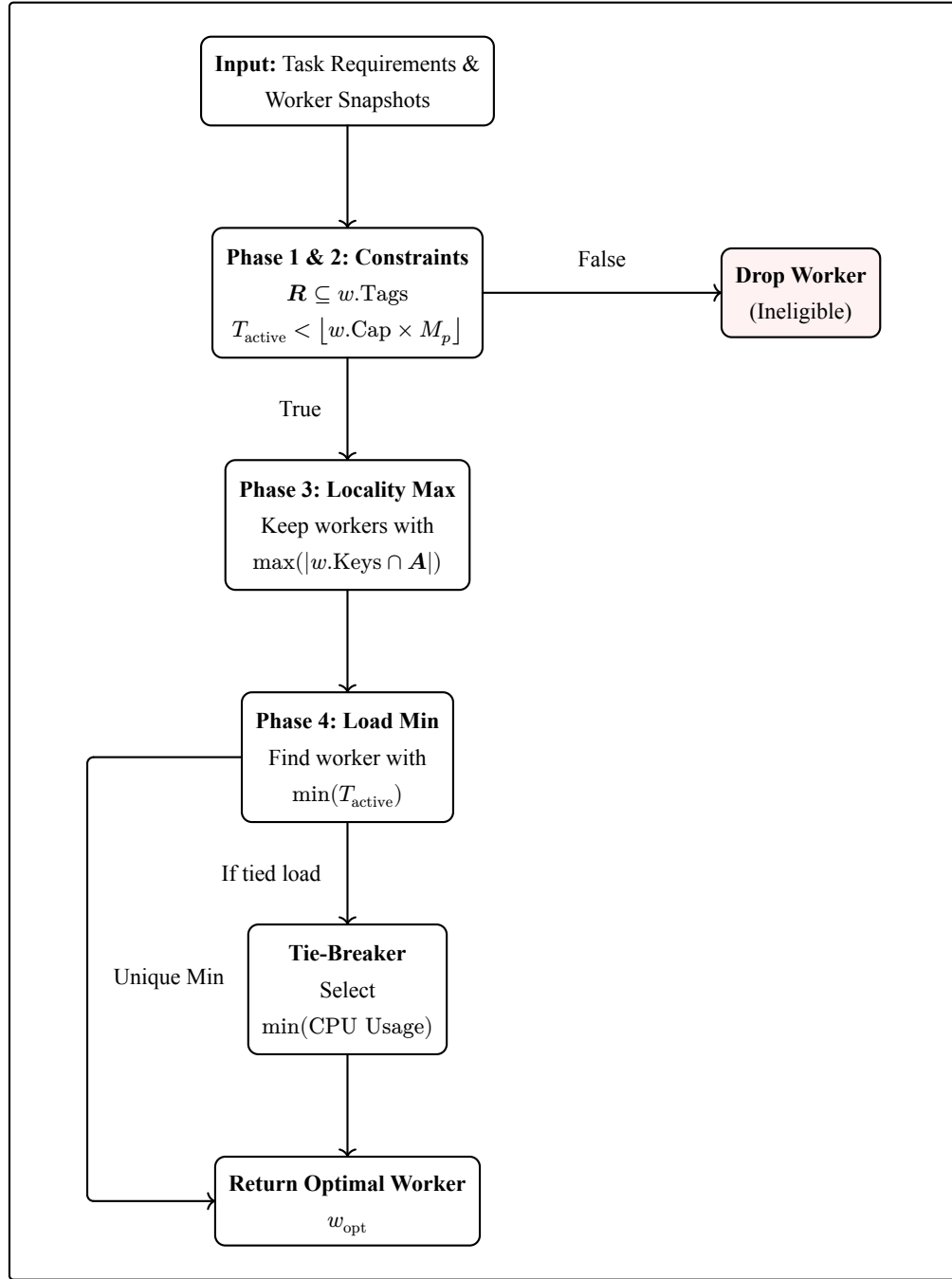


Figure 7: Execution flow and decision matrix of the Least-Loaded routing algorithm.

To formalize the flowchart, the algorithm is implemented sequentially. To prevent visual overflow, it is detailed across two phases: Candidate Formulation (Algorithm 2) and Heuristic Resolution (Algorithm 3).

Input: Required Tags $\mathbf{R}$, Affinity Keys $\mathbf{A}$, Worker snapshots $\mathbf{W}$, Prefetch Multiplier M_p

Output: Localized candidate pool $\mathbf{C}_{\text{local}}$

```

1:  $\mathbf{C} \leftarrow \emptyset$ 
2:
3: // Phase 1 & 2: Hardware Constraints and Capacity Filtering
4: for each worker  $w \in \mathbf{W}$  do
5:    $L_{\text{max}} \leftarrow \max(1, \lfloor w.\text{Capacity} \times M_p \rfloor)$ 
6:   if  $\mathbf{R} \subseteq w.\text{Tags}$  and  $w.\text{ActiveTasks} < L_{\text{max}}$  then
7:      $\mathbf{C} \leftarrow \mathbf{C} \cup \{w\}$ 
8:
9: if  $\mathbf{C} = \emptyset$  then return  $\emptyset$ 
10:
11: // Phase 3: Locality Maximization
12:  $S_{\text{max}} \leftarrow -1$ 
13: for each worker  $w \in \mathbf{C}$  do
14:    $S_w \leftarrow |w.\text{CachedKeys} \cap \mathbf{A}|$ 
15:   if  $S_w > S_{\text{max}}$  then  $S_{\text{max}} \leftarrow S_w$ 
16:
17:  $\mathbf{C}_{\text{local}} \leftarrow \{w \in \mathbf{C} \mid |w.\text{CachedKeys} \cap \mathbf{A}| = S_{\text{max}}\}$ 
18: return  $\mathbf{C}_{\text{local}}$ 

```

Figure 8: Part 1: The $O(N)$ filtering matrix isolating hardware constraints and maximizing data locality.

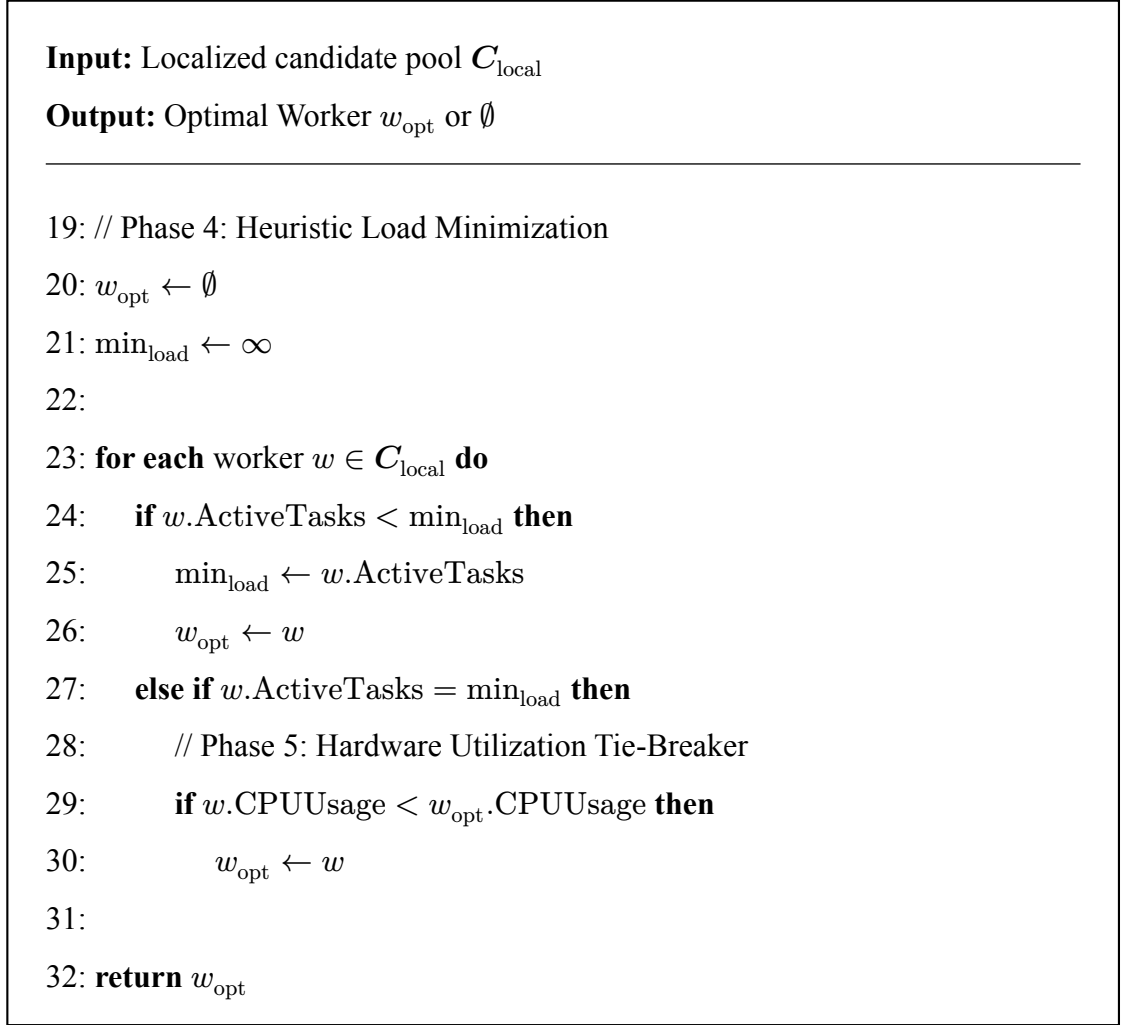


Figure 9: Part 2: Applying the least-loaded heuristic and physical CPU tie-breakers to select the final node.

3.6.5 Algorithmic Complexity Analysis

The architectural decision to detach the `WorkerSnapshot` array from the Coordinator's global `sync.Map` guarantees that the `Pick` algorithm executes purely in CPU cache memory.

For a cluster containing N worker nodes and a task defining K affinity keys:

- The Hardware Constraint and Capacity Filtering phase requires a subset check, operating in $O(N)$ time.
- The Locality Maximization phase iterates through the filtered candidate pool, checking up to K keys, operating in $O(N \times K)$ time.

- The Heuristic Load Minimization phase operates in $O(N)$ time.
- The overall Space Complexity is $O(N)$ to store the temporary candidate pools C and C_{local} .

Because K (the number of affinity keys) is strictly bounded and typically very small (e.g., $K < 5$), the total Time Complexity simplifies asymptotically to $O(N)$. Given modern CPU clock speeds, an $O(N)$ linear scan across an array of a few thousand structs resolves in less than a millisecond. The routing matrix is categorically never the bottleneck.

Algorithm	Time Complexity	Locality Aware	Constraint Aware	Optimal Use Case
Round-Robin	$O(1)$	No	No	Homogeneous clusters processing identical tasks.
Least-Loaded (Base)	$O(N)$	No	Yes	Heterogeneous clusters processing CPU-bound workloads.
Least-Loaded (w/ Affinity)	$O(N \times K)$	Yes	Yes	Heavy I/O and MapReduce workflows (e.g., Matrix Multiplication).

Table 2: Analytical comparison of implemented routing algorithms and their theoretical complexities.

3.7 The Worker Daemon

If the Coordinator is the central nervous system of the architecture, the Worker Daemon is the raw, unthinking muscle. Operating entirely within the Compute Plane, the worker exists solely to catch binary streams, fork isolated OS processes, and return deterministic byte arrays. It possesses absolute zero awareness of the global Directed Acyclic Graph (DAG) topology, nor does it understand the structural semantics of the data it manipulates.

3.7.1 Core Responsibilities and Execution Philosophy

To maintain the absolute decoupling of orchestration and execution, the Worker Daemon adheres to a strict set of responsibilities:

- **Persistent Connection Maintenance:** Bootstrapping a persistent, multiplexed HTTP/2 gRPC connection back to the Control Plane and securing it via Mutual TLS (mTLS) authentication.
- **Telemetry Broadcasting:** Asynchronously transmitting microsecond-accurate CPU saturation and active task loads back to the Coordinator.
- **Bounded-Memory Ingestion:** Reassembling massive partitioned datasets by siphoning incoming chunks directly onto volatile local storage (NVMe/SSD), successfully evading Out-Of-Memory (OOM) kernel panics.
- **POSIX Subprocess Isolation:** Forking native OS processes (`os/exec`) to execute polyglot scripts without the crippling overhead of a JVM wrapper.
- **Upstream Result Streaming:** Capturing the standard output (`stdout`) of the executed subprocesses, buffering it, and piping it back to the Coordinator in controlled 512KB binary frames.

3.7.2 Interface Design and Concurrency Limits

A worker daemon runs on physical, constrained silicon. Dispatching 64 concurrent tasks to a 4-core machine inevitably triggers horrific context-switching, thrashing the CPU cache and destroying throughput. To strictly enforce physical hardware boundaries, we utilize a Counting Semaphore.

```
struct WorkerDaemon {
    id          : String
    cfg          : WorkerConfig
    client       : GrpcWorkerServiceClient
    cpu_cores    : Integer
    active_tasks : AtomicInteger
    semaphore    : Channel [Size: cpu_cores]
    task_mu      : StandardLock
    running      : Map [TaskID → active_task]
}

struct active_task {
    spec          : TaskSpecification
    cancel_func   : ContextCancelFunction
    temp_dir      : String
    start_time    : Timestamp
}
```

The semaphore channel is rigidly bounded to the number of physical CPU cores on the host machine (e.g., `runtime.NumCPU()`). Before the daemon spawns an OS-level subprocess, it must acquire a token. If the semaphore is saturated, incoming tasks simply block in memory. This mathematical guarantee forces a 1:1 active thread-to-core ratio, allowing the physical CPU to execute instructions with maximum cache locality.

3.7.3 Bounded-Memory Data Assembly

As detailed previously, pushing monolithic data payloads across an RPC framework triggers immediate memory exhaustion. To circumvent this, the Coordinator slices payloads into 4MB frames.

When these frames arrive at the Compute Plane, the Worker Daemon must intercept them without accumulating the bytes in RAM. It achieves this via a temporary file-backed `ChunkAssembler`.

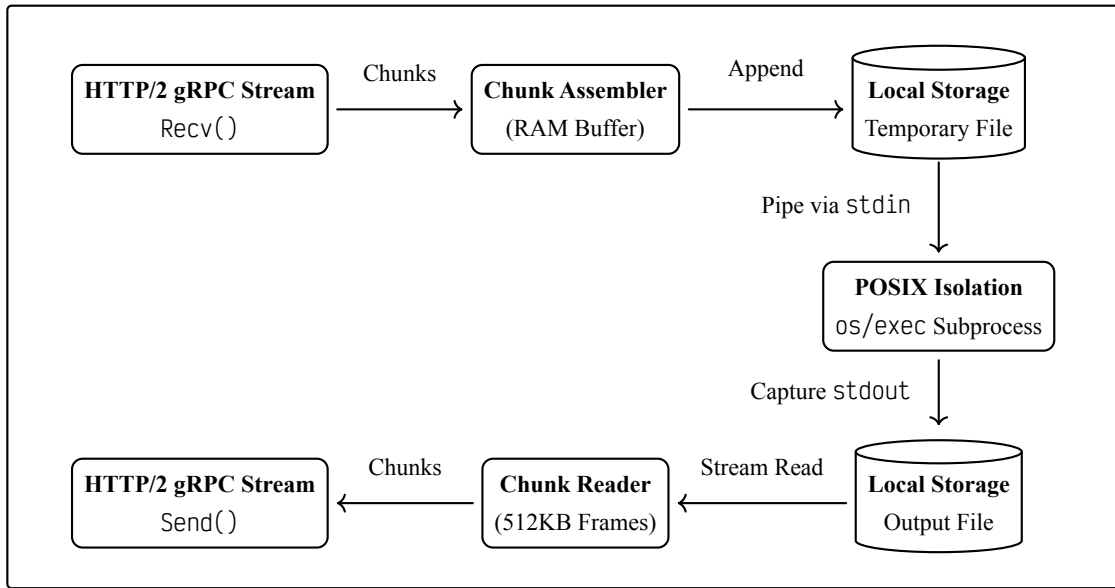


Figure 10: The Worker Daemon’s bounded-memory I/O pipeline. Data is continuously flushed to local disk to preserve RAM during polyglot execution.

By maintaining this strict disk-buffering protocol, a worker node with merely 2GB of physical RAM can trivially process a 50GB dataset partition without suffering a single memory fault.

3.7.4 Polyglot Execution via POSIX Boundaries

Once the binary stream is fully assembled on the worker’s disk, the data crosses the execution boundary. The true power of the Rivage engine lies in its ability to run arbitrary user code without requiring the user to install a heavy middleware SDK.

The daemon leverages the host operating system’s POSIX capabilities. It forks an isolated process (`os/exec.CommandContext`) utilizing the executable path defined in the `TaskSpec`.

To pass the data to this agnostic script, the daemon employs two universal Unix mechanisms:

1. **Standard Input Piping (stdin):** The daemon opens the assembled local file and pipes its byte stream directly into the subprocess’s `stdin`. The user’s script simply reads line-by-line.
2. **Environment Variable Injection:** The daemon injects cluster metadata directly into the POSIX environment space. Variables such as `RIVAGE_TASK_ID` and

`RIVAGE_AFFINITY_KEYS` allow the user's script to dynamically alter its behavior based on the cluster state without requiring external API calls.

Upon successful execution, the script emits its deterministic results to standard output (`stdout`). The daemon captures this stream and writes it to an isolated output file. If the user's script crashes, it emits a non-zero exit code. The daemon intercepts this exit code, halts the capture, and fires a `TASK_STATUS_FAILED` message to the Coordinator, triggering the Scheduler's retry matrix.

3.7.5 The Task Execution Algorithm

The complete lifecycle from chunk ingestion to process execution and upstream transmission is formally defined in Algorithm 4.

Input: A buffered gRPC stream S , System Semaphore Σ

Output: A deterministic binary output stream or execution failure

```

1:  $T_{\text{spec}}, F_{\text{in}} \leftarrow \text{ReassembleFromStream}(S)$ 
2:
4:  $\text{Acquire}(\Sigma)$  // Block until a CPU core is available
5:  $\text{ActiveTasks.Add}(1)$ 
6:
8:  $\text{Ctx} \leftarrow \text{WithTimeout}(\text{Background}(), T_{\text{spec}}.\text{Timeout})$ 
9:  $P \leftarrow \text{exec.CommandContext}(\text{Ctx}, T_{\text{spec}}.\text{Command}, T_{\text{spec}}.\text{Args})$ 
10:
11:  $P.\text{Env} \leftarrow \text{Inject}(T_{\text{spec}}.\text{EnvVars})$ 
12:  $P.\text{Stdin} \leftarrow \text{OpenFile}(F_{\text{in}}, \text{ReadOnly})$ 
13:  $F_{\text{out}} \leftarrow \text{CreateTempFile}()$ 
14:  $P.\text{Stdout} \leftarrow F_{\text{out}}$ 
15:
17:  $E_{\text{err}} \leftarrow P.\text{Run}()$ 
18:
19:  $\text{Release}(\Sigma)$  // Relinquish the CPU core immediately
20:  $\text{ActiveTasks.Sub}(1)$ 
21:
23: if  $E_{\text{err}} \neq \text{null}$  or  $\text{Ctx.Err}() \neq \text{null}$  then
24:    $\text{SendStatus}(S, \text{FAILED})$ 
25:    $\text{CleanupFiles}(F_{\text{in}}, F_{\text{out}})$ 
26:   return
27:
28:  $\text{StreamFileUpstream}(S, F_{\text{out}}, \text{ChunkSize} = 512\text{KB})$ 
29:  $\text{CleanupFiles}(F_{\text{in}}, F_{\text{out}})$ 

```

Figure 11: Pseudocode representing the bounded-memory POSIX execution pipeline within the Worker Daemon.

3.7.6 Telemetry and The Heartbeat Loop

In parallel to the task execution pipeline, the Worker Daemon runs an isolated, high-priority telemetry loop. For the Coordinator's Scheduler to make mathematically sound load-balancing decisions, it requires physical cluster data that is no more than a few milliseconds old.

Every 500 milliseconds, a background thread calculates the sliding window of CPU saturation. Because the connection is HTTP/2 multiplexed, the daemon injects these tiny heartbeat frames directly between the massive 512KB data payload chunks of a returning result. This guarantees telemetric continuity entirely free from TCP Head-of-Line blocking.

3.8 Fault Tolerance and System Resilience

In distributed computing, hardware volatility is a statistical absolute. To maintain exactly-once processing semantics and prevent cluster-wide deadlocks, the Rivage engine implements a rigorous, two-tiered fault architecture, isolating failures across the Compute Plane and the Control Plane.

3.8.1 Compute Plane Resilience (Worker Failures)

Worker daemons operate in an environment where arbitrary, user-defined scripts are executed. These scripts inevitably contain infinite loops and memory leaks. The orchestration engine must guarantee that a catastrophic failure in a worker process does not corrupt the global DAG state.

3.8.1.1 POSIX Isolation and Timeout Enforcement

By leveraging operating system-level process forking, the engine constructs a hard physical boundary. If a user's Python script attempts to allocate 100GB of RAM, the Linux OOM killer violently terminates the child (SIGKILL), but the Go worker daemon survives. The daemon intercepts the non-zero exit status, cleans up the disk, and reports the error upstream.

Furthermore, to prevent an infinite loop, the Coordinator embeds a strict `Timeout` constraint inside every `TaskSpec`. If the system clock exceeds the deadline, the daemon forcefully sends a `SIGTERM` to the child process, instantly reclaiming the CPU core.

3.8.1.2 The Watchdog and Idempotent Retries

If the physical machine loses power, the master's Watchdog catches the stale `last_heartbeat`. Because the MapReduce model demands mathematical idempotency, the master executes zero state rollback. It simply dumps the active tasks back into the `Pending` queue for immediate network re-routing. This guarantees eventual completion without side-effect corruption.

3.8.2 Control Plane Resilience (Master Failures)

The Coordinator is a terrifying single point of failure. External databases, while popular, violate our zero-dependency mandate and inject catastrophic network latency into the dispatch loop.

3.8.2.1 Write-Ahead Logging (WAL) Architecture

To achieve durability with $O(1)$ write latency, we utilize a localized Write-Ahead Log. Append-only sequential writes to an NVMe drive carry virtually zero overhead compared to B-Tree database inserts.

```
struct wal_entry {
    timestamp : Integer // Unix Epoch
    event_type : Byte    // Dispatched=1, Completed=2, Failed=3
    task_id    : String
    worker_id  : String
}

interface JobWAL {
    Log(event: Byte, task: String, worker: String) → Error
    Recover() → []wal_entry
}
```

The master logs `Dispatched` immediately before sending the network payload, and `Completed` the exact millisecond the output hits the disk.

3.8.2.2 The WAL Recovery Algorithm

If the Coordinator crashes midway through a 10,000-task Map stage, it must reconstruct the state upon reboot to avoid repeating successfully completed work. During initialization, it sequentially replays the binary events.

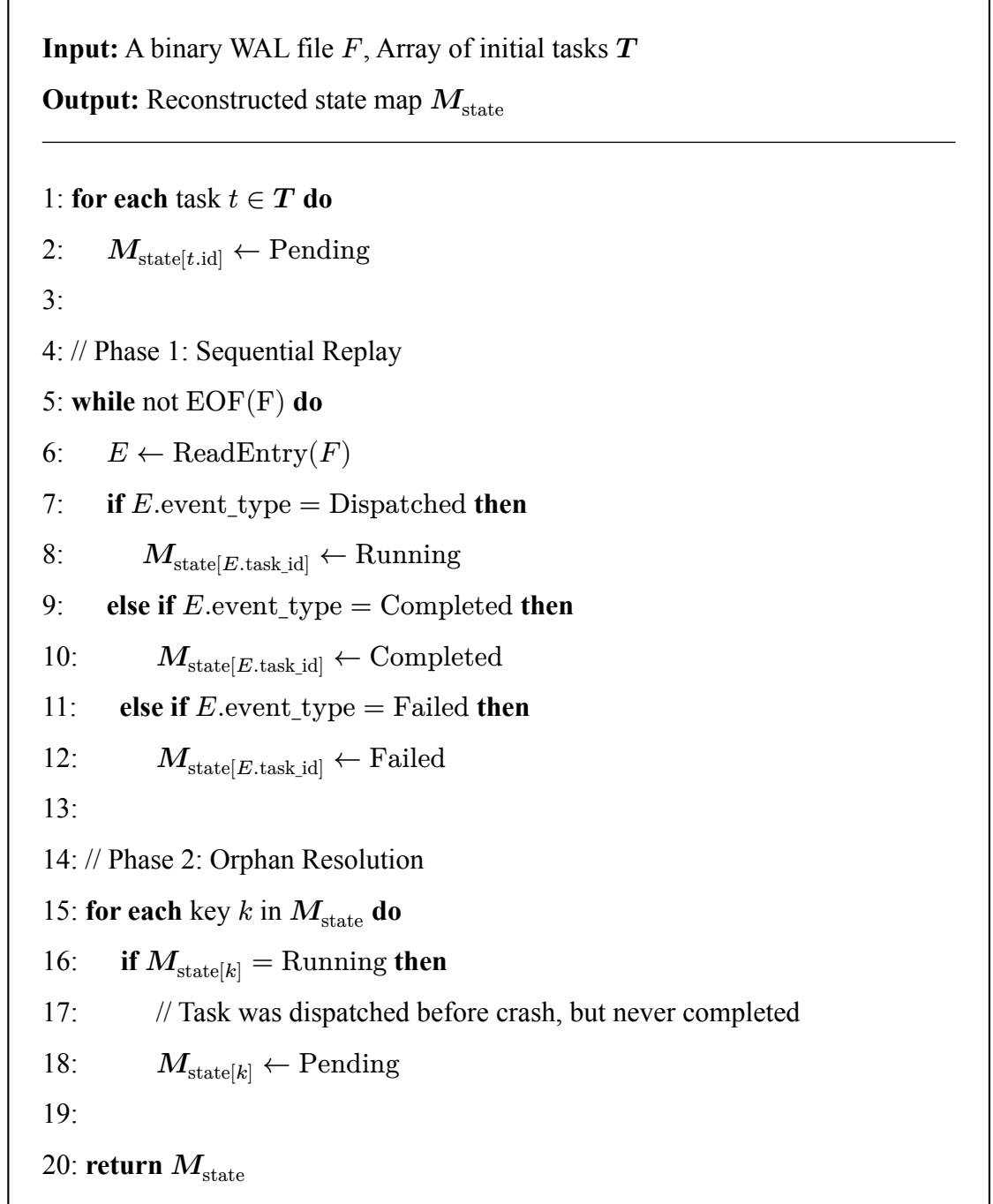


Figure 12: Pseudocode representing the deterministic WAL recovery algorithm executed during master node reboot.

Crucially, in Phase 2 (Orphan Resolution), any task stranded in the `Running` state implies the master dispatched it but crashed before capturing the output. The master conservatively downgrades it back to `Pending`. This structural guarantee ensures that regardless of rolling kernel panics or network partitions, the execution graph will eventually resolve with absolute mathematical integrity.

Failure Scenario	Detection Mechanism	Component Responsible	Recovery Action
Worker OOM Crash	os/exec non-zero exit code	Worker Daemon	Emits <code>FAILED</code> status, aggressively purges temporary disk files.
Worker Hardware Loss	Heartbeat Timeout (Stale Timestamp)	Coordinator Watchdog	Downgrades active tasks to <code>Pending</code> , clears <code>worker_id</code> for network reassignment.
Master Node Power Loss	Boot-time Initialization	Coordinator	Sequentially replays binary <code>.wal</code> file, resolves orphaned running tasks to <code>Pending</code> .
Infinite Loop in Script	Context Deadline Exceeded	Worker Daemon	Sends <code>SIGTERM</code> to subprocess, releases local counting semaphore token.

Table 3: Failure matrix detailing deterministic recovery actions across the Compute and Control planes.

3.9 The Programming Model and Programmable Shuffles

We abstract the horrors of distributed consensus entirely. The user provides only three elements: the polyglot script, the DAG topology, and the routing boundaries.

3.9.1 Polyglot Task Execution

To define a custom Map or Reduce function, a developer simply writes a standalone script. The only contract enforced by the engine is that the script must read its payload from standard input (`stdin`) and emit its deterministic result to standard output (`stdout`).

```
import sys, json

def main():
    // 1. Read grouped intermediate data from the orchestrator
    data = json.load(sys.stdin)

    // 2. Perform custom aggregation logic
    target_key = data["key"]
    total_sum = sum(int(val) for val in data["values"])

    // 3. Emit the final result back to the orchestrator
    print(json.dumps({target_key: total_sum}))

if __name__ == "__main__":
    main()
```

Because this script requires zero Rivage-specific SDKs, it can natively import advanced computational frameworks like `numpy` or `tensorflow` without conflicting with the Go orchestrator.

3.9.2 The DAG Builder Interface

The `dag.Builder` interface within the Go Control Plane allows operators to wire sequential or branching execution stages together effortlessly.

```
pipeline, err := dag.New("distributed_analytics").
    Stage("map_phase",
        dag.ScriptExecutor("python3", "map.py"),
        dag.WithParallelism(100), // Max concurrent tasks
    ).
    Stage("reduce_phase",
        dag.ScriptExecutor("python3", "reduce.py"),
        dag.WithShuffle(dag.JSONKeyGroupShuffle()),
    ).
    Build()
```

When `pipeline.Build()` is invoked, the engine statically analyzes the declared stages, performs Kahn’s Algorithm, and mathematically guarantees the absence of circular dependencies before the network is ever touched.

3.9.3 Programmable Data Routing (ShuffleFunc)

The rigid “Sort and Group-By-Key” paradigm of legacy Hadoop is catastrophic for numerical workloads like distributed matrix multiplication. Rivage solves this by exposing the raw routing boundary via the `ShuffleFunc` interface:

```
type ShuffleFunc func(outputs []TaskOutput) (ShuffleResult, error)
```

By supplying a custom `ShuffleFunc`, the developer overrides the default MapReduce routing. Instead of sorting strings, a developer performing matrix multiplication can write a custom mathematical router:

```
func MatMulRowBandShuffle(outputs []dag.TaskOutput) (dag.ShuffleResult,
error) {
    // 1. Intercept all completed tile multiplications
    rowGroups := make(map[int][]json.RawMessage)

    for _, out := range outputs {
        meta := parseTileMetadata(out.Data)
        // 2. Group matrix tiles mathematically by their geometric Row
        rowGroups[meta.TileRow] = append(rowGroups[meta.TileRow],
out.Data)
    }

    res := make(dag.ShuffleResult)
    for row, tiles := range rowGroups {
        // 3. Generate a custom Reduce task specifically for this row
band
        taskID := fmt.Sprintf("assemble-row-%d", row)
        res[taskID] = dag.TaskInput{Data: serialize(row, tiles)}
    }
    return res, nil
}
```

By injecting a custom mathematical router, the engine dynamically morphs from a standard text-processor into a highly specialized, grid-based High-Performance Computing (HPC) orchestration engine—without installing a single external dependency.

CHAPTER 4

RESULTS AND DISCUSSION

[To be drafted: An empirical evaluation of system performance. We require precise benchmarking metrics detailing the execution of canonical MapReduce workloads (e.g., word count, matrix operations) across heterogeneous clusters. This section must include a rigorous analysis of overall throughput, latency overhead, and peak resource utilization under extreme synthetic stress.]

CHAPTER 5

CONCLUSION AND FUTURE WORKS

[To be drafted: A conclusive synthesis of the architectural findings, emphasizing the demonstrated efficacy of the POSIX-bounded polyglot model. The future scope must aggressively outline the roadmap for implementing advanced load-aware heuristic scheduling algorithms and expanding the I/O capabilities for external, disk-backed shuffling boundaries.]

CHAPTER 5

REFERENCES

- [1] J. Dean and S. Ghemawat, “MapReduce: Simplified Data Processing on Large Clusters,” *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008.
- [2] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, “Spark: Cluster computing with working sets,” in *HotCloud*, 2010, p. 95.
- [3] C. Mohan, D. Haderle, B. Lindsay, H. Pirahesh, and P. Schwarz, “ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging,” *ACM Transactions on Database Systems (TODS)*, vol. 17, no. 1, pp. 94–162, 1992.
- [4] A. D. Birrell and B. J. Nelson, “Implementing Remote Procedure Calls,” *ACM Transactions on Computer Systems (TOCS)*, vol. 2, no. 1, pp. 39–59, 1984.
- [5] gRPC Authors, “gRPC: A high performance, open source universal RPC framework.” [Online]. Available: <https://grpc.io/>
- [6] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proceedings of the April 18-20, 1967, spring joint computer conference*, 1967, pp. 483–485.

CHAPTER 1

APPENDIX A: Output

[To be drafted: Supplementary indices. Compile all raw performance tables, extended Protocol Buffer definitions, and the granular worker-core execution logic that proved too dense for inclusion in the primary manuscript.]